

02. 集成学习

<https://jishuzhan.net/article/1965250835849986049>

<https://weijingmin2000.github.io/files/Data%20Science/Lesson%2013%20Ensemble%20Learning.pdf>

<https://github.com/datawhalechina/ensemble-learning/tree/main>

1. 第一章 集成学习概论

集成学习（Ensemble Learning）是机器学习中一种重要的策略，其核心思想是**将多个学习器组合起来，以获得比单个学习器更好的泛化性能**。俗话说“三个臭皮匠，赛过诸葛亮”，集成学习正是这一思想的体现。

目前比较有代表性的集成方式是 Boosting 和 Bagging。

Boosting 集成是每个基学习器间存在强依赖关系、必须串行生成，目前比较有代表性的工业界框架有 AdaBoost、**GBDT**、**XGBoost** 和 **LightGBM**，其中后三者工业界比较常用；Bagging 集成是每个基学习器间不需要存在依赖关系、可并行化，其中比较具有代表性的是随机森林（即 **Random Forest**），工业也比较常用。

目前业界用的最多的还是 XGB 和 LGBM。

1.1 基本概念

- **基学习器（Base Learner）**：集成中的每个单独模型，也叫“个体学习器”
- **同质集成**：基学习器类型相同（如都是决策树）
- **异质集成**：基学习器类型不同（如决策树 + SVM + 神经网络）
- **强学习器**：最终组合后的模型
- **弱学习器**：性能略优于随机猜测的学习器

基学习器应具备一定的**准确性**（优于随机猜测）和**多样性**（彼此之间存在差异）。



为什么基学习器不推荐使用强学习器？

- **防止过拟合。**强分类器本身已经对训练数据有很强的拟合能力，如果再把多个强分类器组合在一起，极容易**过拟合**训练数据，导致泛化能力下降。**弱分类器**（如决策树桩 Decision Stump）结构简单，单独看虽然欠拟合，但组合后既能提升准确率，又不容易过拟合。
- **多样性。**弱分类器由于能力有限，每个分类器只能捕捉数据的一部分模式，彼此之间**差异较大**，组合后可以形成很好的互补效应。

1.2 两大主流范式

1.2.1 Bagging

核心思想：通过自助采样（Bootstrap）生成多个不同的训练子集，独立训练多个基学习器，最终进行**投票/平均**。

特点：

- 各基学习器**并行训练**，相互独立
- 主要降低**方差（Variance）**，减少过拟合
- 代表算法：**随机森林（Random Forest）**

1.2.2 Boosting（提升法）

核心思想：基学习器**串行训练**，每个新模型重点关注前一个模型预测错误的样本，逐步提升整体性能。

特点：

- 基学习器**串行依赖**
- 主要降低**偏差（Bias）**，提升拟合能力
- 代表算法：AdaBoost、GBDT、XGBoost、LightGBM

1.3 集成学习中的权重

集成学习中，会有多处涉及到权重：

1. 样本权重：通过设置 `sample_weight` 让某些样本权重更重要，影响分裂点的选择 Gini/MSE 计算。AdaBoosting 算法就是通过调整样本权重来迭代树的。
2. 树权重：AdaBoosting 算法最终是累加不同权重的多个树的输出作为最终的输出。
3. 叶子节点权重：叶子节点的权重其实就是树的最终输出。比如，一个树有 5 个叶子节点，每个叶子节点都有一个权重，一个输入 x 命中叶子节点 1，那么这颗树的输出就是叶子节点 1 的权重。

💡 “每棵树重新生长结构 + 计算叶子权重” 是所有基于决策树的 Boosting 算法（包括 GBDT、XGBoost、LightGBM、CatBoost）的标准流程。

2. 第二章 Bagging 之随机森林

随机森林既可以胜任分类任务又可以胜任回归任务。属于判别式模型。

欲得到泛化性能强的集成，集成中的个体学习器应尽可能相互独立；虽然个体学习器完全独立在现实任务中无法做到，但可以设法使基学习器尽可能具有较大的差异。

给定一个训练数据集，进行反复采样，可产生出若干个不同的子集，再从每个数据子集中训练出一个基学习器。同时，为获得很好的集成，个体学习器不能太差，如果采样出的每个子集都完全不同，则每个基学习器只用到了一小部分训练数据，不足以有效学习。所以反复地有放回采样（bootstrap sampling）是一种有效的方式，同时可产生相互有交叠的采样子集。

2.1 基本概念

随机森林算法的名称含义如下：

- **森林**：指的是由多棵 **决策树（Decision Tree）** 组成的集合。
- **随机**：指的是在构建每一棵决策树时，算法会引入两种随机性，确保每棵树都与众不同。

这两种随机性就是 **Bootstrap（有放回采样）** 与 **Aggregating（聚合）**。

最终，对于分类任务，森林通过 **投票（多数决）** 给出结果；对于回归任务，则通过 **取平均值** 给出结果。

随机森林算法流程图



2.2 Bootstrap（有放回采样）

采样过程描述如下：

给定包含 m 个样本的数据集，我们先随机取出一个样本放入采集中，再把该样本放回初始数据集，使得下次采样时该样本仍有可能被选中，这样，经过 m 次随机采样操作，我们得到含 m 个样本的采样集。

由有放回采样的性质得：初始训练集中约有 63.2%（1-32.8%）的样本出现在采样集中。

解释：样本在 m 次采样中始终不被采到的概率是 $(1 - \frac{1}{m})^m$ ，取极限得到

$$\lim_{m \rightarrow +\infty} (1 - \frac{1}{m})^m = \frac{1}{e} \approx 0.368。$$

2.3 调参项

可以使用 `scikit-learn` 库中的随机森林模型，有以下调参项：

参数名	含义	典型值/影响	通俗解释
n_estimators	森林中决策树的数量。	默认 100。值越大，模型通常越稳定，性能越好，但计算成本也越高。	"委员会的人数"。人越多，决策通常越可靠，但开会时间也更长。
max_depth	单棵决策树的最大深度。	默认 None（不限制）。限制深度可以防止过拟合，使模型更简单。	"限制每个人的发言时间"。防止某个专家（树）钻牛角尖，过度关注训练数据的细节。
max_features	寻找最佳分裂时考虑的特征数。	可以是整数、浮点数或 'auto' / 'sqrt'。这是引入"特征随机性"的关键参数。	"每次讨论只随机看几个方面"。确保每棵树从不同角度分析问题，增加多样性。
min_samples_split	节点分裂所需的最小样本数。	默认 2。值越大，树生长越保守，越不容易过拟合。	"一个小组至少要有几个人才能继续分组讨论"。避免因为一两个样本就创建一个新规则。
min_samples_leaf	叶节点所需的最小样本数。	默认 1。值越大，模型越平滑。	"最终结论至少需要基于几个案例"。确保每个结论都有一定的数据支撑。
bootstrap	是否使用 Bootstrap 采样。	默认 True。如果设为 False，则将使用整个数据集训练每棵树，但会失去一部分随机性。	"是否允许一个人重复发言"。开启就是 Bagging 的精髓。

2.4 分类与回归任务

在 sklearn 的随机森林算法中：

- 如果做分类任务，需要使用 RandomForestClassifier，底层创建的是 DecisionTreeClassifier，决策树一般以 Gini 指数当作分裂标准。
- 如果做回归任务，需要使用 RandomForestRegressor，底层创建的是 DecisionTreeRegressor，决策树一般以均方差作为分裂标准。

在预测时，变量 estimators_ 会存储每一棵树的结果：

代码块

```
1 # 伪代码示意
2 forest.# 伪代码示意
```

```
3 forest.estimators_ = [tree_0, tree_1, tree_2, ..., tree_99] = [tree_0, tree_1, tree_2, ..., tree_99]
```

如果是分类任务，`estimators_` 存的是标签，最后取占比最高的标签当作最终的输出；如果是回归任务 `estimators_` 存的是数值，最终取平均值当作预测值。

2.5 实验分析

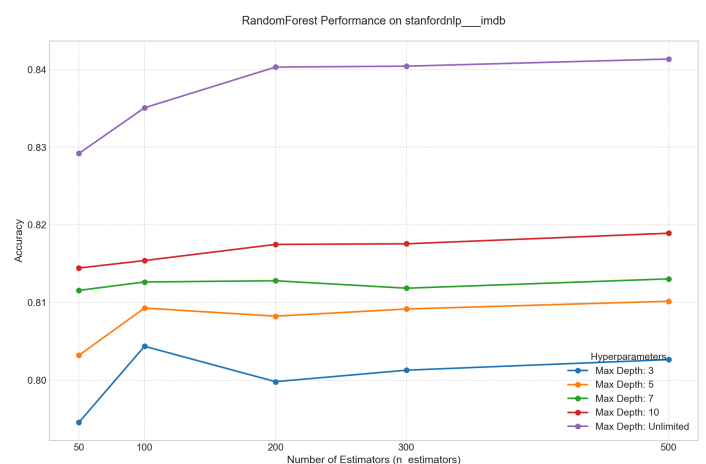
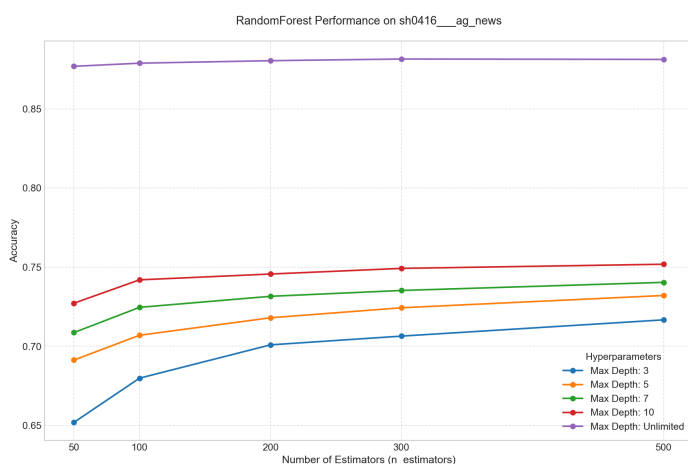
代码仓库可见：<https://github.com/Unagi-cq/th-nlp-learning/tree/main>

主要代码如下：

代码块

```
1 # 输入特征 数据集采用AGNEWS和IMDB
2 vectorizer = TfidfVectorizer(max_features=2000, stop_words='english')
3 X_train_vec = vectorizer.fit_transform(X_train)
4 X_test_vec = vectorizer.transform(X_test)
5
6 # 模型
7 model = RandomForestClassifier(
8     n_estimators=n_estimators,
9     max_depth=max_depth,
10    n_jobs=32,
11    random_state=42,
12    verbose=0 # 减少输出噪音
13 )
14 model.fit(X_train_vec, y_train)
15 y_pred = model.predict(X_test_vec)
16 acc = accuracy_score(y_test, y_pred)
```

数据集采用的是 AGNEWS 和 IMDB，实验主要探究不同的树个数和深度对实验准确率的影响，实验结果如下：



根据实验结果可以得出一些简单的结论：

- 树深度越深，数据集的准确率就越高。
- 树的个数大到一定程度之后，边际收益就会变低，但是这个平衡点在不同的数据集、不同的树深度下有所区别，需要调参来分析。
- AG News 数据集是四分类，IMDB 数据集是情感二分类。根据图中的效果，可以发现随机森林算法在 IMDB 数据集二分类任务上，即使树深度、树个数很小也可以得到不错的效果，这是因为任务简单，很容易找到决策边界。但是在 AG News 数据集需要一定的参数配置才能到达 0.85 的效果。
- 在随机森林文本分类中，**树的深度决定模型能不能学到足够复杂的文本特征组合，树的数量决定集成结果是否稳定**。在这两个任务上，深度的影响明显强于树数量的影响。

3. 第三章 Boosting 之 AdaBoosting

3.1 基本思想

AdaBoost (Adaptive Boosting) 有自适应集成之意。对于 AdaBoost 而言，有两个重点：一是在每一轮如何改变训练数据的权值；二是如何将弱分类器组合成一个强分类器。

关于第一个问题，AdaBoost 的做法是，提高那些被前一轮弱分类器错误分类样本的权值，而降低那些被正确分类样本的权值，这样一来，那些没有得到正确分类的数据，由于其权值的加大而受到后一轮的弱分类器的更大关注，于是，分类问题被一系列的弱分类器“分而治之”。

至于第二个问题，即弱分类器的组合，AdaBoost 采取加权多数表决的方法，具体地，加大分类误差率小的弱分类器的权值，使其在表决中起较大的作用，减小分类误差率较大的弱分类器的权值，使其在表决中起较小的作用。AdaBoost 的巧妙之处就在于它将这些想法自然且有效地实现在一种算法里。

Boosting 本质上是一种“元算法”或“框架思想”，它定义了如何组合多个模型、如何迭代更新权重（或拟合残差），但并不限制底层基学习器的具体类型。

只要基学习器满足一个核心条件：比随机猜测略好（即准确率 $> 50\%$ ，称为“弱学习器”），理论上都可以作为 Boosting 的基组件。

对于底层基学习器，理论上我们可以任意替换，如 LR、KNN、决策树等。但是对于若干个基学习器每迭代一轮后，如何做优化，是 Boosting 需要考虑的问题。具体来说，**AdaBoosting 明确了下面两个问题：**

1. 在每一轮后，提高那些被前一轮弱分类器错误分类的样本权重，降低那些被前一轮弱分类器成功分类的样本权重

2. 对于如何组合多个弱分类器，AdaBoosting 采用加权多数表决的方法，加大分类误差率小的弱分类器的权值，使其在表决中起较大的作用，减小分类误差率大的弱分类器的权值，使其在表决中起较小的作用。

AdaBoosting 是早期基于样本调整权重的方法。

3.2 算法描述

(1) 初始化训练数据的权值分布

$$D_1 = (w_{11}, \dots, w_{1i}, \dots, w_{1N}), \quad w_{1i} = \frac{1}{N}, \quad i = 1, 2, \dots, N$$

(2) 对 $m = 1, 2, \dots, M$

- (a) 使用具有权值分布 D_m 的训练数据集学习，得到基本分类器

$$G_m(x) : \mathcal{X} \rightarrow \{-1, +1\}$$

- (b) 计算 $G_m(x)$ 在训练数据集上的分类误差率

$$e_m = P(G_m(x_i) \neq y_i) = \sum_{i=1}^N w_{mi} I(G_m(x_i) \neq y_i)$$

- (c) 计算 $G_m(x)$ 的系数

$$\alpha_m = \frac{1}{2} \log \frac{1 - e_m}{e_m}$$

- (d) 更新训练数据集的权值分布

$$\begin{aligned} D_{m+1} &= (w_{m+1,1}, \dots, w_{m+1,i}, \dots, w_{m+1,N}) \\ w_{m+1,i} &= \frac{w_{mi}}{Z_m} \exp(-\alpha_m y_i G_m(x_i)), \quad i = 1, 2, \dots, N \\ Z_m &= \sum_{i=1}^N w_{mi} \exp(-\alpha_m y_i G_m(x_i)) \end{aligned}$$

(3) 构建基本分类器的线性组合

$$f(x) = \sum_{m=1}^M \alpha_m G_m(x)$$

得到最终分类器

$$G(x) = \text{sign}(f(x)) = \text{sign} \left(\sum_{m=1}^M \alpha_m G_m(x) \right)$$

以上是 AdaBoosting 方法的数学描述。

下面我们通过代码来理解。

代码块


```

1  def fit(self, X, y):
2      """
3      训练AdaBoost模型
4      :param X: (n_samples, n_features) 的输入数据
5      :param y: (n_samples,) 的标签, 取值为 +1 或 -1
6      """
7      n_samples = X.shape[0]
8
9      # 初始化样本权重, 均匀分布
10     weights = np.ones(n_samples) / n_samples
11
12     for i in range(self.n_estimators):
13         # 训练弱学习器
14         model = self.trainer.fit(X, y, sample_weight=weights)
15
16         # 预测
17         preds = model.predict(X)
18
19         # 计算加权错误率
20         miss = (preds != y).astype(int)
21         error = np.dot(weights, miss) / np.sum(weights)
22
23         # 计算弱学习器权重
24         #  $\alpha = 1/2 * \ln((1 - error) / error)$ 
25         if error == 0:
26             alpha = 10.0 # 防止除零
27         else:
28             alpha = self.learning_rate * 0.5 * np.log((1 - error) / error)
29
30         # 更新样本权重
31         #  $weights = weights * \exp(-\alpha * y * preds)$ 
32         weights = weights * np.exp(-alpha * y * preds)
33         weights = weights / np.sum(weights) # 归一化
34
35         self.models.append(model)
36         self.model_weights.append(alpha)
37
38         # 如果错误率接近0, 提前结束
39         if error < 1e-10:
40             break

```

3.3 分类与回归任务

在 sklearn 的 `AdaBoosting` 算法中:

- 如果做分类任务，需要使用 `AdaBoostClassifier`，它继承 `BaseWeightBoosting`，底层创建的是 `DecisionTreeClassifier`，训练时会传递样本权重，决策树一般以 Gini 指数当作分裂标准。
- 如果做回归任务，需要使用 `AdaBoostRegressor`，它继承 `BaseWeightBoosting`，底层创建的是 `DecisionTreeRegressor`，训练时会传递样本权重，决策树一般以均方差作为分裂标准。

`BaseWeightBoosting` 封装了一些通用流程，而具体的差异（如：如何计算误差、如何计算 α 、如何预测）则由子类去实现。

1. 初始化样本权重。
2. 循环训练弱学习器。
3. 计算弱学习器的误差/得分。
4. 更新样本权重。
5. 计算弱学习器的投票权重（ α ）。

在预测时：

- 如果是分类任务，因为每一棵树输出的是正 1/负 1 标签，最后取所有树的加权值，为正则输出 +1，为负则输出 -1。
- 回归任务一般采用加权中位数作为输出值，算法相对来说复杂一点。对于每一棵树的输出，升序排列，对应的权重也升序排列，然后从最小的预测值开始累加权重，直到累加值超过总权重的一半，此时对应的预测值即为最终结果。

3.4 实验分析

代码仓库可见：<https://github.com/Unagi-cq/th-nlp-learning/tree/main>

主要代码如下：

代码块

```
1  # 输入特征 数据集采用AGNEWS和IMDB
2  vectorizer = TfidfVectorizer(max_features=2000, stop_words='english')
3  X_train_vec = vectorizer.fit_transform(X_train)
4  X_test_vec = vectorizer.transform(X_test)
5
6  # 模型
7  def train_once(X_train_vec, X_test_vec, y_train, y_test, n_estimators,
8                max_depth):
9      """
10     训练单次 AdaBoost 模型并返回准确率
11     """
```

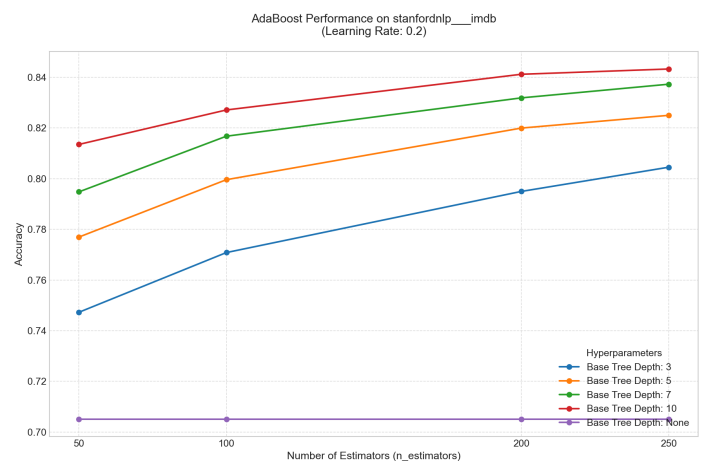
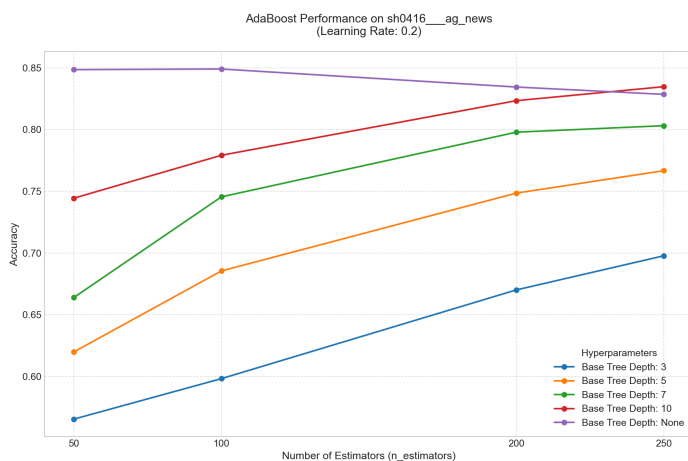
```

11     # 基分类器：决策树，深度由网格搜索决定
12     base_estimator = DecisionTreeClassifier(
13         max_depth=max_depth,
14         random_state=42
15     )
16
17     model = AdaBoostClassifier(
18         estimator=base_estimator,
19         n_estimators=n_estimators,
20         learning_rate=FIXED_LEARNING_RATE,
21         random_state=42
22     )
23
24     model.fit(X_train_vec, y_train)
25     y_pred = model.predict(X_test_vec)
26     acc = accuracy_score(y_test, y_pred)
27     return acc

```

数据集采用的是 AGNEWS 和 IMDB，实验主要探究不同的树个数和深度对实验准确率的影响，实验结果如下：

实验时间非常非常久。AdaBoosting 难以充分利用 CPU，并且是串行训练的模型，非常慢。



根据实验结果可以得出一些简单的结论：

- 随着 `n_estimators` 的增加，多数配置下模型准确率均呈现上升趋势，说明集成迭代能够有效提升分类性能。
- 在 IMDB 数据集上，深度为 10 的基决策树取得最佳效果，准确率最高约为 0.843，说明情感分类任务需要更强的非线性表达能力。相比之下，AG News 数据集上 `max_depth=None` 在较少迭代次数下取得较高准确率，但随着迭代次数增加性能下降，表现出一定过拟合趋势。
- 综合来看，`max_depth=10` 在两个数据集上均表现较为稳定，能够在模型复杂度与泛化能力之间取得较好平衡。

因此，AdaBoost 的性能不仅受到迭代次数影响，也高度依赖基学习器复杂度，过浅的树容易欠拟合，而完全生长的树则可能导致过拟合。

4. 第四章 Boosting 之 GBDT

GBDT (Gradient Boosting Decision Tree, **梯度提升决策树**) 是一种基于 Boosting 思想的集成学习算法，通过迭代地训练一系列弱学习器（通常是决策树），并将它们组合起来形成一个强学习器。

4.1 基本概念

GBDT 算法的名称含义如下：

- **提升**：指的是串行地训练模型，每一棵新树都旨在纠正前一棵树的错误（即拟合残差）。
- **梯度下降**：在函数空间中进行优化，不同于传统的 Adaboost 直接调整样本权重，GBDT 利用损失函数的负梯度作为当前模型的“残差”近似值，让新树去拟合这个负梯度。

不是直接拟合残差，而是把损失函数的负梯度当作残差。



残差、梯度与损失？

- 残差也就是误差，等于预测值与真实值之间的差距。
- 梯度是修正的方向，是让错误变小的**方向**提示（如参数应该加还是减）。
- 损失是把残差转化为优化目标，方便我们通过数学形式优化。

4.2 GBDT 算法

4.2.1 回归任务

假设最终模型是：

$$F(x) = f_1(x) + f_2(x) + f_3(x) + \cdots + f_M(x)$$

其中每一个 f 都是一棵决策树。

训练过程大致如下：

第一步，初始化一个简单模型。

对于回归任务，通常可以用所有标签的平均值作为初始预测：

$$F_0(x) = \bar{y}$$

第二步，计算当前模型的误差方向，也就是负梯度：

$$r_i = -\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)}$$

第三步，训练一棵新的 CART 回归树去拟合这些 r_j 。

第四步，把新树加入到整体模型中：

$$F_m(x) = F_{m-1}(x) + \eta f_m(x)$$

η 是学习率，用来控制每棵树对最终结果的贡献。

第五步，重复训练很多棵树，直到达到指定树数量，或者模型收敛。

4.2.2 分类任务

GBDT 分类 = 用一棵棵回归树，不断修正当前样本属于某一类的预测分数，最后再把分数转成概率。

假设有训练集：

$$(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$$

第一步，初始化模型，先给所有样本一个初始分数。

如果正样本比例是 $\bar{y}$ ，初始分数通常可以设为：

$$F_0(x) = \log \frac{\bar{y}}{1 - \bar{y}}$$

第二步，计算当前预测概率。

第 m 轮时，当前模型为：

$$F_{m-1}(x)$$

把它转成概率：

$$p_i = \frac{1}{1 + e^{-F_{m-1}(x_i)}}$$

第三步，计算负梯度。对于每个样本，计算：

$$r_i = y_i - p_i$$

这个 r_j 就是这一轮新树要学习的目标。

第四步，训练一棵回归树。用样本特征去拟合负梯度。

第五步，更新模型。把新树加入原模型。

$$F_m(x) = F_{m-1}(x) + \eta h_m(x)$$

第六步，重复训练很多棵树。

$$F_M(x) = F_0(x) + \eta h_1(x) + \eta h_2(x) + \cdots + \eta h_M(x)$$

最后将最终分数转成概率：

$$p(x) = \frac{1}{1 + e^{-F_M(x)}}$$

再根据阈值进行分类。

4.2.3 GBDT 的优点

GBDT 对表格数据非常强，尤其是在传统机器学习任务中，经常能取得很好的效果。它可以自动处理非线性关系和特征组合，不需要像线性模型那样手工设计很多交叉特征。

GBDT 对特征尺度不敏感，不需要像 SVM、KNN、神经网络那样特别依赖归一化或标准化。

它还能输出特征重要性，方便解释模型。

因此在很多结构化数据场景中，GBDT 系列模型长期都是强基线模型。

4.2.4 GBDT 的缺点

GBDT 里面的树是一棵接一棵训练的，所以原始 GBDT 难以完全并行，训练速度通常比随机森林慢。

同时，GBDT 对参数比较敏感，比如树的数量、树深、学习率、子采样比例等设置不合理时，容易过拟合。

另外，普通 GBDT 对高维稀疏特征不一定特别友好，比如超高维文本 one-hot 特征场景，线性模型或深度学习方法有时更合适。

4.3 重要参数怎么理解？

GBDT 常见参数有几个：

n_estimators：树的数量。树越多，模型表达能力越强，但训练更慢，也更容易过拟合。

learning_rate：学习率。控制每棵树的贡献。学习率小，训练更稳，但通常需要更多树。

max_depth：树的最大深度。树越深，单棵树越复杂，越容易过拟合。

min_samples_split / min_samples_leaf：控制节点继续划分的条件，可以防止树过于复杂。

subsample: 每轮训练时抽取一部分样本，类似随机性，可以缓解过拟合。

一个常见经验是：

较小 learning_rate + 较多 n_estimators

通常比：

较大 learning_rate + 较少 n_estimators

更稳定，但训练时间更长。

4.4 分类与回归任务

Sklearn 中的 `GradientBoostingClassifier` 和 `GradientBoostingRegressor` 都使用 `DecisionTreeRegressor` 作为基学习器。

这是因为 `DecisionTreeRegressor` 的叶子节点输出的是该节点内样本目标值的平均值（或经过牛顿法修正后的最佳值）。这正好可以用来逼近那些连续的梯度值。`DecisionTreeClassifier` 的叶子节点输出的是类别标签或概率分布，它无法直接输出一个用于累加的连续梯度值。

在预测时：

- 如果是回归任务，那么先取训练集的标签平均值作为起点，然后一直累加每一棵树的输出，最终的结果就是预测值。
- 如果是分类任务，计算初始对数几率 $F_0 = \ln\left(\frac{p}{1-p}\right)$ ，其中 p 是正样本在训练集中的比例。然后也是依次累加每一棵树的输出，最终把结果经过 sigmoid，得到的输出则为正例的概率。

4.5 实验分析

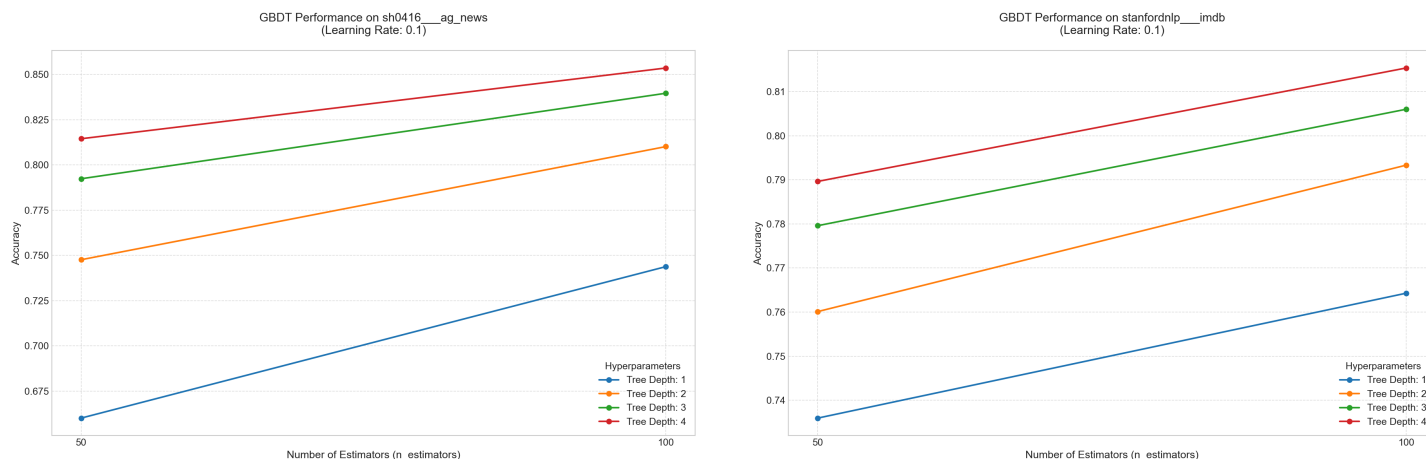
代码仓库可见：<https://github.com/Unagi-cq/th-nlp-learning/tree/main>

主要代码如下：

代码块

```
1 model = GradientBoostingClassifier(  
2     n_estimators=n_estimators,  
3     learning_rate=FIXED_LEARNING_RATE,  
4     max_depth=max_depth,  
5     random_state=42,  
6 )  
7 model.fit(X_train_vec, y_train)  
8 y_pred = model.predict(X_test_vec)  
9 acc = accuracy_score(y_test, y_pred)
```


数据集采用的是 AGNEWS 和 IMDB，实验主要探究不同的树个数和深度对实验准确率的影响，实验结果如下：



根据实验结果可以得出一些简单的结论：

- 树深度越深，数据集的准确率就越高。

由于运行时间实在是太久，这里就简单做了几个小参数试验。

5. 第五章 Boosting 之 XGBoost

XGBoost (eXtreme Gradient Boosting) 是一款非常流行的集成学习算法模型，它是 DBGT 的优化和升级，底层仍然使用 CART 回归树，但是增加了二阶导、正则化、并行等特性支持。

5.1 核心思想

XGBoost 属于 **Boosting** 集成学习方法，其本质是**串行**训练一系列弱学习器（通常是 CART 回归树），每一棵树都试图拟合前一棵树的残差（或负梯度）。

- **目标函数：**

XGBoost 的目标函数由两部分组成：**损失函数 + 正则化项**。

$$\mathcal{L}(\phi) = \sum_i l(\hat{y}_i, y_i) + \sum_k \Omega(f_k)$$

其中：

- l 是衡量预测值 $\hat{y}_i$ 与真实值 y_i 差异的损失函数（如平方损失、对数损失等）。
- $\Omega(f_k)$ 是第 k 棵树的复杂度控制项，用于防止过拟合。这是 XGBoost 相比传统 GBDT 的重要改进之一。

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda ||w||^2$$

- T : 叶子节点个数。
- w : 叶子节点的权重向量。
- γ, λ : 正则化参数。

这是一个非常深刻且切中要害的问题。要理解 XGBoost 如何利用二阶导数优化，我们需要从 **GBDT 的局限性** 出发，通过数学推导对比两者的差异。

5.2 XGB 与 GBDT 对比

- **GBDT**: 使用**一阶泰勒展开**近似损失函数，本质上是用**负梯度**（残差）来拟合下一棵树。它只利用了损失函数的“斜率”信息。
- **XGBoost**: 使用**二阶泰勒展开**近似损失函数，同时利用**一阶导数（梯度 g ）**和**二阶导数（海森矩阵/曲率 h ）**。它不仅知道“往哪走”（方向），还知道“走多大步”以及“当前点的弯曲程度”（步长/置信度）。

5.2.1 GBDT：一阶近似

在 GBDT 的第 t 轮迭代中，希望找到一棵树 $f_t(x)$ ，使得加入这棵树后的总损失最小：

$$\mathcal{L}^{(t)} = \sum_{i=1}^n l(y_i, \hat{y}_i^{(t-1)} + f_t(x_i))$$

由于 l 可能是任意复杂的非线性函数直接求解困难。GBDT 采用**一阶泰勒展开**进行近似：

$$l(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) \approx l(y_i, \hat{y}_i^{(t-1)}) + g_i f_t(x_i)$$

其中 $g_i = \frac{\partial l(y_i, \hat{y})}{\partial \hat{y}} \Big|_{\hat{y}=\hat{y}^{(t-1)}}$ 是一阶导数（梯度）。

忽略常数项 $l(y_i, \hat{y}_i^{(t-1)})$ ，GBDT 的目标变成了最小化：

$$\sum_{i=1}^n g_i f_t(x_i)$$

GBDT 试图让 $f_t(x_i)$ 与 $-g_i$ 越接近越好（因为如果 $f_t(x_i) = -g_i$ ，则 $g_i f_t(x_i) = -g_i^2$ ，是一个较大的负数，从而降低损失）。

所以，GBDT 的每一步就是在拟合**负梯度**（对于平方损失，负梯度就是残差 $y - \hat{y}$ ）。

5.2.2 XGBoost：二阶近似

XGBoost 对损失函数进行**二阶泰勒展开**：

$$l(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) \approx l(y_i, \hat{y}_i^{(t-1)}) + g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i)$$

其中：

- $g_i = \partial_{\hat{y}} l(y_i, \hat{y}_i^{(t-1)})$ （一阶导数）
- $h_i = \partial_{\hat{y}}^2 l(y_i, \hat{y}_i^{(t-1)})$ （二阶导数）

同样忽略常数项，XGBoost 在第 t 轮的目标函数近似为：

$$\tilde{\mathcal{L}}^{(t)} = \sum_{i=1}^n \left[g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i) \right] + \Omega(f_t)$$

求解叶子节点权重

假设树的结构已确定（即每个样本 i 被映射到叶子节点 j ，记为 $q(x_i) = j$ ），且叶子节点的权重为 w_j 。那么 $f_t(x_i) = w_{q(x_i)}$ 。

将正则化项 $\Omega(f_t) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2$ 代入，目标函数可以按叶子节点重组，令 $G_j = \sum_{i \in I_j} g_i$ ， $H_j = \sum_{i \in I_j} h_i$ 。解得：

$$w_j^* = -\frac{G_j}{H_j + \lambda}$$

代入得到最小损失（用于评估分裂增益）：

$$\tilde{\mathcal{L}}^{(t)}(q) = -\frac{1}{2} \sum_{j=1}^T \frac{G_j^2}{H_j + \lambda} + \gamma T$$

5.3 XGB的优势

1. **更精确的梯度方向**：二阶信息提供了损失函数的局部曲率，使得每次更新的权重 w_j^* 更接近真实的最优解，而不是仅仅沿着梯度方向走一小步。
2. **自适应学习率**：公式 $w_j^* = -\frac{G_j}{H_j + \lambda}$ 中， H_j 起到了自适应调节学习率的作用。不同叶子节点、不同迭代阶段，步长都是动态计算的。
3. **更好的分裂评估**：分裂增益公式 $\frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right] - \gamma$ 同时考虑了梯度总和与黑塞矩阵总和，能更准确地将“难分样本”和“易分样本”区分开，找到信息量最大的分裂点。



不同于XGB，深度学习模型的反向传播与梯度更新主要涉及一阶导数。

标准 SGD（随机梯度下降）及其变体（Adam, RMSprop, Momentum 等）都是一阶优化算法。它们只利用损失函数的“斜率”信息来决定参数更新的方向和大小。

XGBoost 是 GBDT 的工程化极致实现，其核心优势在于：

1. **精度更高**：二阶导数 + 正则化。
2. **速度更快**：并行计算 + 稀疏优化。
3. **灵活性更强**：支持自定义损失函数和评估指标。

5.4 实验分析

代码仓库可见：<https://github.com/Unagi-cq/th-nlp-learning/tree/main>

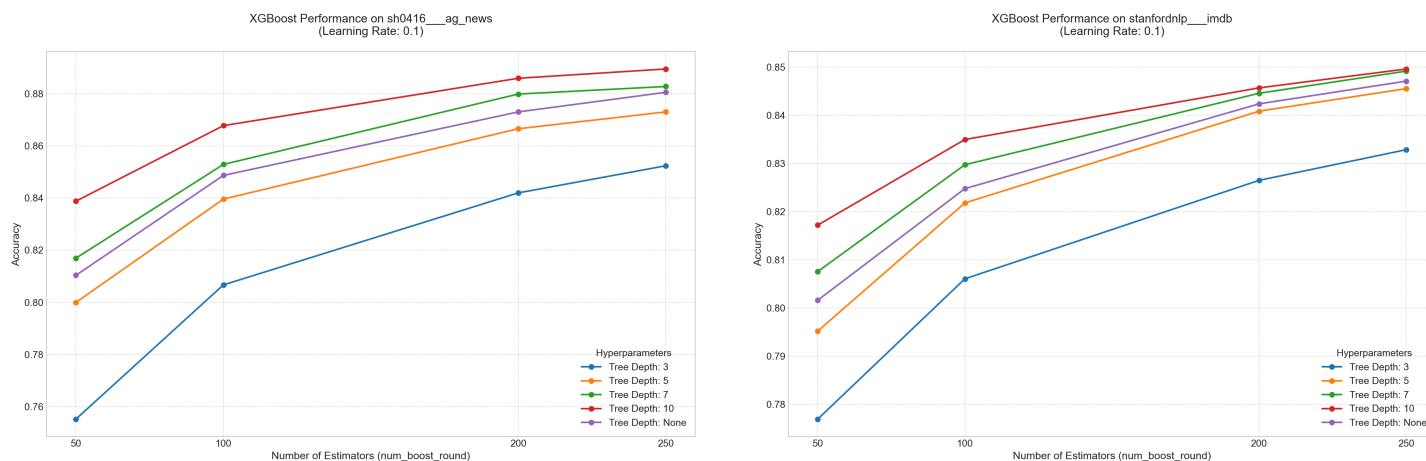
主要代码如下：

代码块

```
1  def build_params(num_class, max_depth):
2      params = {
3          "tree_method": "hist",
4          "device": "cpu",
5          "max_depth": max_depth,
6          "learning_rate": FIXED_LEARNING_RATE,
7      }
8
9      if num_class == 2:
10         params.update({
11             "objective": "binary:logistic",
12             "eval_metric": "logloss",
13         })
14     else:
15         params.update({
16             "objective": "multi:softprob",
17             "num_class": num_class,
18             "eval_metric": "mlogloss",
19         })
20
21     return params
22
23 def train_once(dtrain, dtest, y_test, num_class, n_estimators, max_depth):
24     params = build_params(num_class, max_depth)
25     model = xgb.train(params, dtrain, num_boost_round=n_estimators)
26     y_pred = predict_labels(model, dtest, num_class)
```

```
27     acc = accuracy_score(y_test, y_pred)
28     return acc
```

数据集采用的是 AGNEWS 和 IMDB，实验主要探究不同的树个数和深度对实验准确率的影响，实验结果如下：



根据实验结果可以得出一些简单的结论：

- 在所有设置下，随着树的数量（Estimators）增加，模型准确率均呈现上升趋势。这符合 Boosting 算法的特性：通过串行添加弱学习器来逐步降低偏差（Bias）。
- 曲线的斜率随着树数量的增加而逐渐变缓。例如，从 50 到 100 棵树的提升幅度，明显大于从 200 到 250 棵树的提升幅度。这说明模型在初期快速拟合主要模式，后期则在微调残差，收敛速度变慢。

6. 第六章 Boosting 之 LightGBM

LightGBM是一个专为效率、可扩展性和准确性而设计的开源机器学习框架。它是由微软开发的一款分布式机器学习工具包的一部分，于2017年4月24日发布。

LightGBM是在现有的梯度提升决策树（如XGBoost）的基础上构建的，它提供了以下概述的几个关键改进。

6.1 LightGBM的改进

与XGBoost模型相比，LightGBM的训练速度显著更快。这是通过诸如基于梯度的单侧采样（GOSS）等技术实现的，该技术专注于误差较高的数据点，从而减少了不必要的计算。

LightGBM利用直方图进行高效的树构建。与其他GBDT算法相比，减少了内存消耗，使其特别适合处理传统方法可能无法装入内存的大规模数据集。

LightGBM设计为可扩展的。由于其具有分布式学习能力，因此可以有效地处理大规模数据集。这允许在多台机器上并行训练模型，从而显著加快大规模数据集的处理速度。

在优先考虑速度和内存效率的同时，LightGBM并没有在准确性上妥协。所采用的技术旨在实现与以前的GBDT模型相似甚至更好的性能。

6.2 LGBM与XGB的优劣

XGBoost 和 LightGBM 都是极其优秀的梯度提升树（GBDT）算法，没有绝对的“谁更好”，它们各有侧重。XGBoost 在预测精度和稳定性上通常稍占优势，而 LightGBM 在训练速度和内存占用上遥遥领先。

以下是两者的具体特性对比，可根据业务场景按需选择：

6.2.1 精度与效果

- XGBoost：采用按层生长（Level-wise）策略，不容易过拟合，且由于发展时间较长，调参生态成熟，在各类机器学习竞赛（如 Kaggle）和工业界中，通常能取得略高或更稳的准确率。
- LightGBM：采用叶子生长（Leaf-wise）策略，容易收敛，但这也导致它更容易过拟合，需要通过精细设置参数（如调小 `max_depth` 和 `num_leaves`）来控制复杂度。

6.2.2 速度与资源

- LightGBM：专为大数据集设计，训练速度极快，内存占用低。通过基于梯度的单边采样（GOSS）和互斥特征捆绑（EFB）技术，在大规模高维数据下效率优势明显。
- XGBoost：速度同样很快，但相比 LightGBM 稍显逊色，内存消耗通常也较大。

6.2.3 数据特征处理

- XGBoost：内置了对缺失值的自动处理机制，对于稀疏数据有天然的优势，特征处理更加省心。
- LightGBM：同样支持缺失值和类别特征处理，但在高维类别特征较多时，LightGBM 的原生处理速度非常快，效果也更好。

6.2.4 选型建议

- 首选 LightGBM：当数据量极大（如千万级样本）、特征维度极高，或者开发周期紧迫需要快速迭代训练时，它是首选。
- 首选 XGBoost：当追求极致的准确率，或者处理中小型数据集时，XGBoost 通常是更稳健、更容易调出高精度的选择。

6.3 实验分析

代码仓库可见：<https://github.com/Unagi-cq/th-nlp-learning/tree/main>

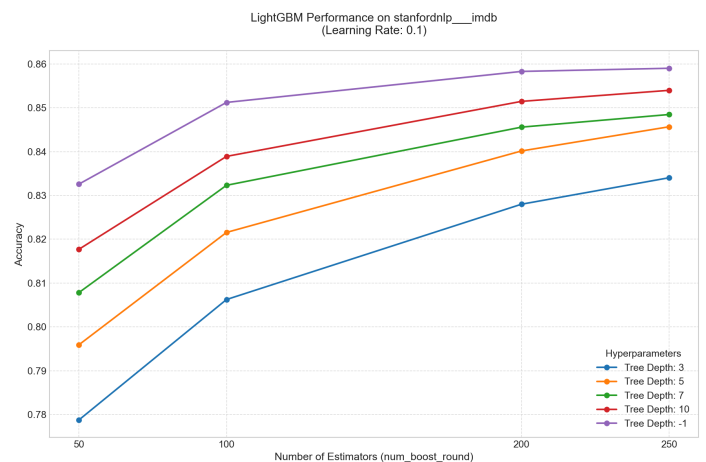
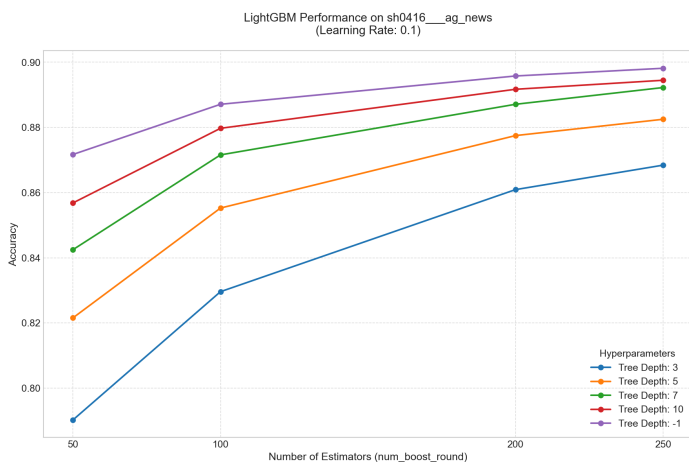
主要代码如下：

```

1  def build_params(num_class, max_depth):
2      params = {
3          "max_depth": max_depth,
4          "learning_rate": FIXED_LEARNING_RATE,
5          "verbose": -1,
6      }
7
8      if num_class == 2:
9          params.update({
10             "objective": "binary",
11             "metric": "binary_logloss",
12         })
13     else:
14         params.update({
15             "objective": "multiclass",
16             "num_class": num_class,
17             "metric": "multi_logloss",
18         })
19
20     return params
21
22 def train_once(train_data, X_test_vec, y_test, num_class, n_estimators,
23               max_depth):
24     params = build_params(num_class, max_depth)
25     model = lgb.train(params, train_data, num_boost_round=n_estimators)
26     y_pred = predict_labels(model, X_test_vec, num_class)
27     acc = accuracy_score(y_test, y_pred)
28     return acc

```

数据集采用的是 AGNEWS 和 IMDB，实验主要探究不同的树个数和深度对实验准确率的影响，实验结果如下：



根据实验结果可以得出一些简单的结论：

- 在所有设置下，随着树的数量（Estimators）增加，模型准确率均呈现上升趋势。这符合 Boosting 算法的特性：通过串行添加弱学习器来逐步降低偏差（Bias）。
- 曲线的斜率随着树数量的增加而逐渐变缓。例如，从 50 到 100 棵树的提升幅度，明显大于从 200 到 250 棵树的提升幅度。这说明模型在初期快速拟合主要模式，后期则在微调残差，收敛速度变慢。

7. 第七章 总结

对于相同数据集和相近的决策树配置，随机森林通常具有更高的训练并行性，因为其各棵树之间相互独立，可以并行构建；而 Boosting 系列算法需要逐轮根据上一轮模型的误差、残差或样本权重更新后续学习器，因此整体训练过程具有较强的串行依赖，训练速度往往相对较慢。

XGBoost、LightGBM是最常用且效果最好的模型。

但在 XGBoost、LightGBM 等工程化实现中，算法会通过特征并行、数据并行、直方图优化和 GPU 加速等方式缓解这一问题。

- 问题1：为什么XGB不需要特征标准化，而 DNN 需要特征归一化？

树模型是非参数化的、基于规则的模型，它不计算特征的加权求和，因此特征尺度对其毫无影响。标准化反而增加了预处理开销，毫无必要。

DNN 需要特征归一化/标准化是因为加速收敛、防止梯度爆炸/消失 & 激活函数饱和。

- 问题2：集成学习模型支持多分类、多标签文本分类吗？

模型	原生支持多分类 (Multi-class)?	原生支持多标签 (Multi-label)?	如何实现多标签？
Random Forest	是	否	MultiOutputClassifier 或 ClassifierChain
AdaBoost	是	否	MultiOutputClassifier
GBDT (sklearn)	是	否	MultiOutputClassifier
XGBoost	是	否	训练 KK 个二分类模型 (One-vs-Rest)
LightGBM	是	否	

核心结论：没有任何一个主流的梯度提升树或随机森林算法“原生”直接输出多标签结果。它们都是为单目标（Single Target）设计的。

但是，所有这些模型都可以通过 Wrapper（包装器）策略轻松实现多标签分类。